Software Quality Project

Hannover Weather Statistics

SoSe 2026

Project Description

The objective of the project is to complete and test a small existing Java program, with a particular emphasis on the principles of software quality discussed during the seminar.

The chosen Java program is the *Hannover Weather Statistics* (hws) application, which provides simple weather statistics for Hannover (Campus Linden). After the user specifies a start and end date via the console, the program retrieves the hourly recorded temperatures for the selected time period from the Open-Meteo historical weather REST API (https://open-meteo.com/en/docs/historical-weather-api), computes several basic statistical measures, and displays the results on the console. The calculated statistics include minimum, maximum, mean, and median temperature, as well the percentage of how much of the data is below 0 degrees (cold) and how much is above 30 degrees (hot). An example of input and output could look as follows:

```
Hannover Weather Statistics:
Start date (yyyy-mm-dd): 2024-01-01
End date (yyyy-mm-dd): 2024-12-31
Min. Temperature: -8.6 °C
Max. Temperature: 33.1 °C
Mean Temperature: 11.6 °C
Median Temperature: 11.3 °C
Cold: 3.4 %
Hot: 0.1 %
```

Download

A copy of the Java/Maven project can be obtained from the following Git repository:

https://gitlab.gwdg.de/dennis.chong/hws2026.git

General

The provided Java/Maven project represents an unfinished implementation. Currently, only access to the Open Meteo REST API has been pre-implemented within the **de.hsh.hws.openmeteo.OpenMeteoService** class. For instructional purposes, the implementation of this class has been kept intentionally simple, relying solely on native Java functionality. Within the scope of this project, this class is to be treated as a given component and must not be modified. The only public method of the class retrieves the hourly temperature data for the specified time range from the Open Meteo REST API and return results as a plain double array:

```
public double[] getTemperature(LocalDate startDate, LocalDate endDate)
```

Each day is represented by 24 values. If a value is unavailable for some reason, `NaN` is inserted into the array. This situation occurs in particular when the selected period includes the current day, for which data concerning future hours is naturally not yet available.

The class **`de.hsh.hws.app.App`** serves as the project's designated entry point. At present, only the basic mechanism for reading the start and end dates has been outlined. All remaining functionality of the application remains to be developed.

Project Tasks

- Students are required to implement the missing functionalities of the program. This includes appropriate handling of user input, retrieval of weather data via the `OpenMeteoService`, and computation as well as presentation of the statistical measures mentioned above. All computed values must consistently be rounded to one decimal place. Proper error handling for both syntactic and semantic input errors must also be implemented.
- Students are expected to ensure good modularization and high testability of their code, and to apply all clean code principles discussed during the seminar. Finally, they should use a static code analysis tool of their choice to identify potential issues. The results should be documented and interpreted, including any corrective measures derived from them.
- Students are required to systematically design, perform, and document appropriate test cases for manual testing of their solution. The test cases should be meaningful, representative, and as comprehensive as possible to effectively detect any defects.
- Students are required to systematically design, implement, and document suitable test cases for automated unit testing. The test cases should be meaningful, representative, and as comprehensive as possible to effectively detect any defects. All unit tests must be implemented using JUnit. Where appropriate, suitable mocks should be created with Mockito. The code coverage of the test code should be measured using a tool of choice. The results should be documented and interpreted, including any measures derived from them. Note that the OpenMeteoService class is considered a given component and does not require test coverage.
- Students are expected to assess the performance of their implementation to identify any bottlenecks in the program (if any). If necessary, a dedicated test program should be developed to execute the core logic in a controlled manner, allowing reliable performance measurements. The results must be documented and interpreted, including any actions or optimizations derived from them.

Submission Guidelines

- The completed solution must be provided in a report via email. The report should be PDF file consisting of max 25 text pages. It is important to ensure that all key aspects of the solution are adequately addressed and discussed in the report. A copy of the full (formatted) source code is to include as appendix in the report. The appendix is not subject to the page limit. The source code will **not** be pushed back to the original Git repository.
- Any use of artificial intelligence tools must be clearly disclosed, including a description of their purpose and extent of use. Students remain fully responsible for the submitted source code and must demonstrate complete understanding of all components and all concepts of their solution. If needed, additional questions may be asked during a walkthrough or oral examination.
- Submission deadline: **June 9, 2026**.